

B.TECH 1ST YEAR · C PROGRAMMING

Unit V

Functions & File Handling in C

Programming for Problem Solving

Complete Exam-Ready Study Module

Functions

Pointers in Functions

Recursion

Scope & Lifetime

Storage Classes

File Handling

Arrays as Parameters



Covers: 14 Sections · Diagrams · Programs · Q&A · Quick Revision

Textbooks: Kernighan & Ritchie | Balagurusamy | Forouzan

CONTENTS

Table of Contents

1. Introduction to Functions	3
2. Function Declaration and Definition	4
3. Function Calling Mechanism	5
4. Types of Functions	6
5. Parameter Passing	7
6. Pointers in Functions	8
7. Arrays as Function Parameters	9
8. Scope of Variables	10
9. Lifetime of Variables	11
10. Storage Classes	12
11. Introduction to File Handling	13
12. File Operations in C	14
13. Questions & Answers	15–19
14. Quick Revision Sheet	20

Syllabus Coverage Map

Topic	Sub-Topics	Section
Functions in C	Intro, Declaration, Definition, Call, Return, Types	1–4
Advanced Functions	Pointers as parameters, Arrays as parameters	5–7
Scope & Lifetime	Local, Global, Static, Storage Classes	8–10
File Handling	Introduction, Operations, Modes, Programs	11–12
Exam Prep	2-mark & 5-mark Q&A, Programs	13
Revision	Quick reference, Syntax summary	14

 **Reference Textbooks:**

- 1. Kernighan & Ritchie – *The C Programming Language*, Prentice Hall (1988)
- 2. Gottfried – *Schaum's Outline of Programming with C*, McGraw-Hill (1996)
- 3. Balagurusamy – *Computing Fundamentals and C Programming*, McGraw-Hill (2008)

4. Rema Theraja – *Programming in C*, Oxford (2016)

5. Forouzan, Gilberg, Prasad – *C Programming: A Problem Solving Approach*, Cengage (3rd Ed.)

SECTION 01

Introduction to Functions

DEFINITION

A **function** in C is a self-contained block of code that performs a specific task. It is a named, reusable unit of logic that can be invoked (called) from anywhere in the program. Functions are the building blocks of modular programming.

1.1 Why Functions?

Without functions, programs would consist of one long sequence of statements — difficult to read, debug, and maintain. Functions solve this by dividing large programs into smaller, manageable subunits. Each function encapsulates a specific responsibility.

- **Modularity:** Breaks complex programs into simpler pieces.
- **Reusability:** Write once, call many times from different locations.
- **Readability:** Self-documenting code through meaningful function names.
- **Maintainability:** Changes in logic need only be made in one place.
- **Debugging:** Errors are isolated to specific functions.
- **Abstraction:** Callers need not know *how* a function works, only what it does.

1.2 Structure of a C Program with Functions

C PROGRAM STRUCTURE

```
/* Preprocessor directives */
#include <stdio.h>

/* Function prototype (declaration) */
int add(int a, int b);

/* Main function - entry point */
int main() {
    int result = add(5, 3); /* Function call */
    printf("Sum = %d\n", result);
    return 0;
}

/* Function definition */
int add(int a, int b) {
    return a + b;
}
```

1.3 Function Call Stack – Diagram

FIG 1.1 – FUNCTION CALL STACK EXECUTION



1.4 Advantages and Applications of Functions

✓ Advantages

- Promotes code reuse
- Reduces program length
- Simplifies debugging
- Enables team development
- Improves program structure

🔧 Applications

- Mathematical computations
- Sorting & searching
- Input/output operations
- Library function creation
- Event-driven programming

SECTION 02

Function Declaration and Definition

2.1 Function Declaration (Prototype)

DEFINITION

A **function declaration** (also called a prototype) tells the compiler about the function's name, return type, and the types of its parameters *before* the function is actually defined. It does not contain the function body.

SYNTAX – FUNCTION DECLARATION

```
return_type function_name(parameter_type1 param1, parameter_type2 param2, ...);
```

Examples:

```
int add(int a, int b); /* Returns int, takes two ints */
float area(float radius); /* Returns float, takes one float */
void display(); /* Returns nothing, no parameters */
void greet(char name[]); /* Takes a character array */
```

Note: Parameter names in prototypes are optional; only types are required. E.g., `int add(int, int);` is valid.

2.2 Function Definition

DEFINITION

A **function definition** provides the actual body (implementation) of the function. It contains the code that executes when the function is called.

SYNTAX – FUNCTION DEFINITION

```
return_type function_name(parameter_type1 param1, parameter_type2 param2) {
    /* local variable declarations */
    /* statements */
    return expression; /* optional for void functions */
}
```

2.3 Complete Example – Function with Declaration and Definition

C PROGRAM – AREA OF CIRCLE USING FUNCTION

```
#include <stdio.h>
#define PI 3.14159

/* Function Declaration (Prototype) */
float circleArea(float r);
```

```

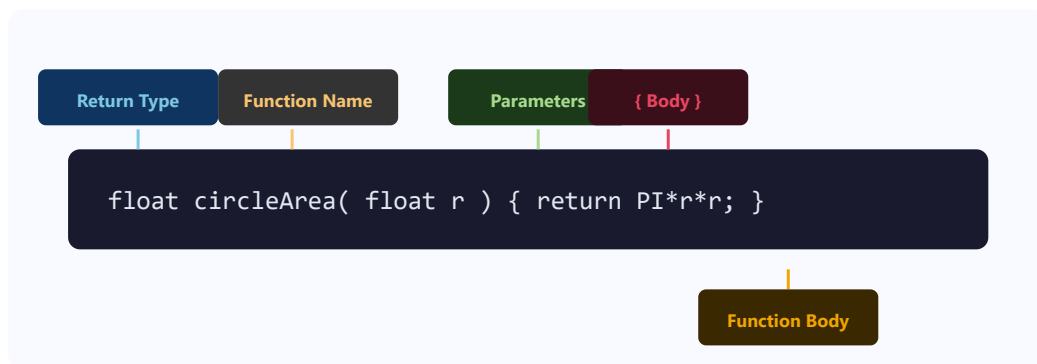
int main() {
    float radius, area;
    printf("Enter radius: ");
    scanf("%f", &radius);
    area = circleArea(radius); /* Function call */
    printf("Area = %.2f\n", area);
    return 0;
}

/* Function Definition */
float circleArea(float r) {
    return PI * r * r;
}

```

2.4 Anatomy of a Function – Diagram

FIG 2.1 – PARTS OF A FUNCTION



2.5 Key Rules

- A function must be **declared before it is called**, or its definition must appear before the call.
- The return type must match the declared type; use void if nothing is returned.
- Function names follow identifier naming rules (no spaces, starts with letter or `_`).
- In C, functions cannot be defined inside other functions (no nested function definitions).

SECTION 03

Function Calling Mechanism

DEFINITION

A **function call** is the statement that transfers control to a function. When the function completes, control returns to the statement immediately after the call. This process involves saving the return address, passing arguments, executing the function body, and returning the result.

3.1 Syntax of a Function Call

SYNTAX

```
variable = function_name(argument1, argument2, ...);
```

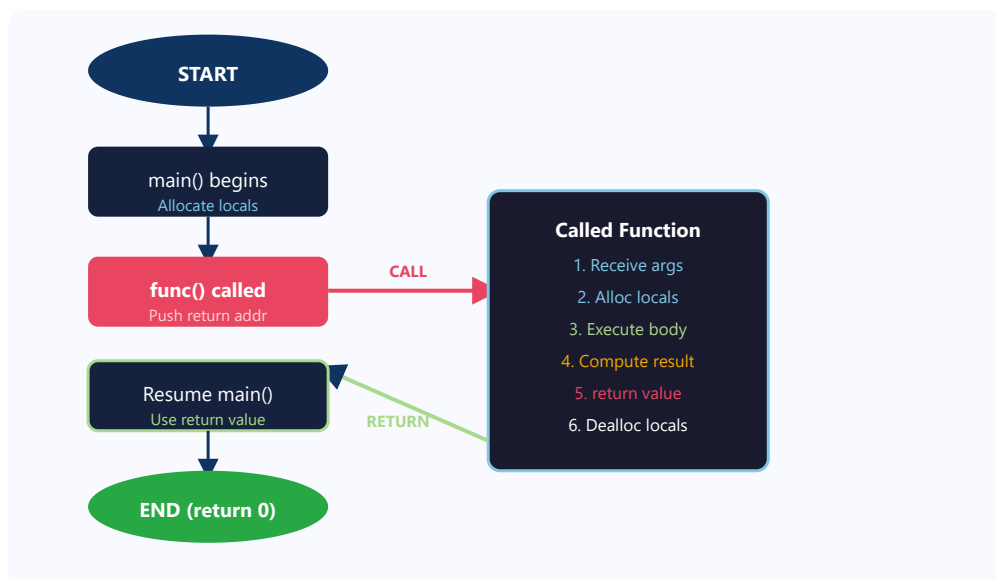
```
/* OR without return value: */
```

```
function_name(argument1, argument2);
```

3.2 Step-by-Step Execution Process

1. The **calling function** (e.g., main) encounters the function call.
2. The **return address** (the next instruction's address) is pushed onto the call stack.
3. **Arguments** are evaluated and copies are placed on the stack (for call by value).
4. Control jumps to the **called function's code**.
5. Local variables of the called function are **allocated on the stack**.
6. The function **executes** its statements.
7. The **return value** (if any) is placed in a register or on the stack.
8. Control returns to the **caller**; local variables of the called function are destroyed.

3.3 Function Execution Flow – Diagram

FIG 3.1 – FUNCTION EXECUTION FLOW DIAGRAM

3.4 Actual Arguments vs. Formal Parameters

Feature	Actual Arguments	Formal Parameters
Location	In the function call	In the function definition
Example	<code>add(5, 3)</code>	<code>int add(int a, int b)</code>
Purpose	Supply values to function	Receive values from caller
Scope	Caller's scope	Function's local scope

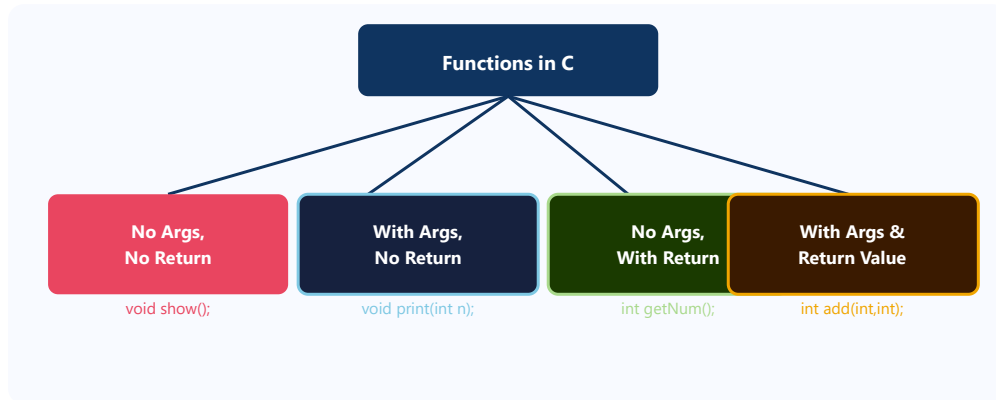
SECTION 04

Types of Functions

Functions in C are classified based on whether they have **arguments** and whether they **return values**.

4.1 Classification Overview

FIG 4.1 – TYPES OF FUNCTIONS IN C



4.2 Type 1 – No Arguments, No Return Value

```
void greet() {  
    printf("Hello, Welcome to C Programming!\n");  
}  
  
int main() { greet(); return 0; }
```

4.3 Type 2 – With Arguments, No Return Value

```
void printSquare(int n) {  
    printf("Square of %d = %d\n", n, n * n);  
}  
  
int main() { printSquare(5); return 0; }
```

4.4 Type 3 – No Arguments, With Return Value

```
int getNumber() {  
    int n;  
    printf("Enter a number: ");  
    scanf("%d", &n);  
    return n;  
}  
  
int main() { int x = getNumber(); printf("%d\n", x); return 0; }
```

4.5 Type 4 – With Arguments and Return Value

```
int add(int a, int b) {
    return a + b;
}

int main() {
    printf("Sum = %d\n", add(10, 20));
    return 0;
}
```

4.6 Comparison Table

Type	Arguments	Return Value	Use Case	Example Prototype
Type 1	None	None (void)	Display menus, greetings	void show();
Type 2	Yes	None (void)	Print values, perform I/O	void print(int);
Type 3	None	Yes	Read user input	int input();
Type 4	Yes	Yes	Computations, calculations	int add(int,int);

4.7 Recursive Functions

DEFINITION

A function that calls itself is called a **recursive function**. Recursion must have a *base condition* to prevent infinite calls.

FACTORIAL USING RECURSION

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1; /* Base condition */
    return n * factorial(n - 1); /* Recursive call */
}

int main() {
    printf("5! = %d\n", factorial(5)); /* Output: 5! = 120 */
    return 0;
}
```

SECTION 05

Parameter Passing (Call by Value vs. Call by Pointer)

5.1 Call by Value

DEFINITION

In **call by value**, a copy of the actual argument is passed to the function. Changes made to the parameter inside the function do *not* affect the original variable.

CALL BY VALUE – SWAP ATTEMPT (DOES NOT WORK)

```
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("Inside: a=%d, b=%d\n", a, b);
}

int main() {
    int x=10, y=20;
    swap(x, y);
    printf("After: x=%d, y=%d\n", x, y); /* x=10, y=20 - unchanged! */
    return 0;
}
```

5.2 Parameter Passing Diagram – Call by Value vs. Pointer

FIG 5.1 – CALL BY VALUE VS CALL BY POINTER



5.3 Comparison Table

Feature	Call by Value	Call by Pointer
What is passed	Copy of the value	Address (memory location)
Original modified?	No	Yes (via dereferencing)

Memory usage	Extra memory for copy	No copy needed
Safety	Safe – original protected	Risky – can modify original
Speed	Slower for large data	Faster (only address passed)
Syntax in call	func(x)	func(&x)
Parameter declaration	int a	int *a

SECTION 06

Modifying Parameters Using Pointers

DEFINITION

A **pointer** is a variable that stores the memory address of another variable. When a pointer is passed to a function, the function can directly access and modify the original variable through *dereferencing* (using the `*` operator).

6.1 Key Pointer Operators

Operator	Name	Meaning	Example
<code>&</code>	Address-of	Gets memory address of variable	<code>&x</code> → address of x
<code>*</code>	Dereference	Accesses value at pointer's address	<code>*p</code> → value at address p

6.2 Pointer-Based Swap – Working Program

```
C PROGRAM – SWAP USING POINTERS (WORKS CORRECTLY)

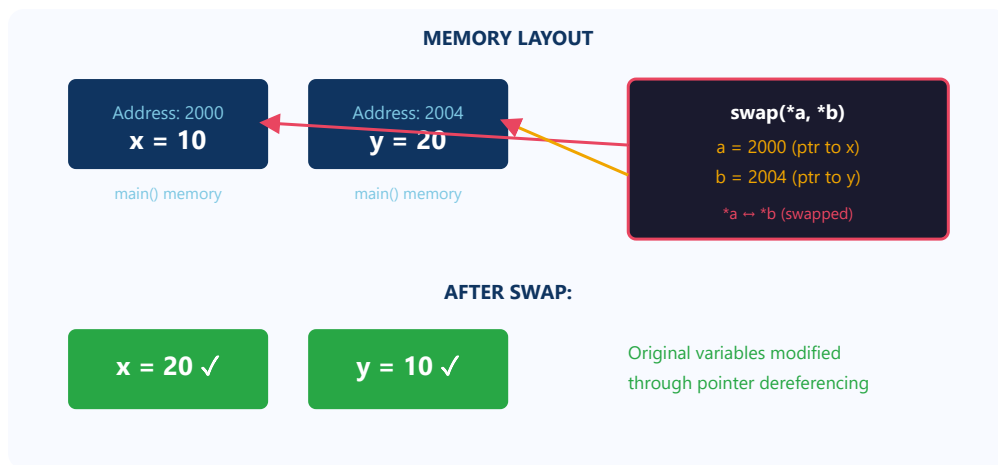
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a; /* Store value at address a */
    *a = *b; /* Place value at b into a's location */
    *b = temp; /* Place old a value into b's location */
}

int main() {
    int x = 10, y = 20;
    printf("Before: x=%d, y=%d\n", x, y);
    swap(&x, &y); /* Pass addresses */
    printf("After: x=%d, y=%d\n", x, y); /* x=20, y=10 */
    return 0;
}
```

6.3 Memory Diagram – Pointer Modification

FIG 6.1 – POINTER-BASED FUNCTION MODIFICATION



6.4 Returning Multiple Values Using Pointers

C PROGRAM – SUM AND DIFFERENCE USING POINTERS

```
#include <stdio.h>

void compute(int a, int b, int *sum, int *diff) {
    *sum = a + b;
    *diff = a - b;
}

int main() {
    int s, d;
    compute(15, 5, &s, &d);
    printf("Sum=%d Diff=%d\n", s, d); /* Sum=20 Diff=10 */
    return 0;
}
```

SECTION 07

Arrays as Function Parameters

DEFINITION

When an **array** is passed to a function, the *base address* (address of the first element) is passed. This means the function receives a pointer to the array and can directly modify the original array — unlike call by value for simple variables.

Important: Arrays are *always* passed by reference (base address) in C — there is no copy of the array made. Changes inside the function affect the original array.

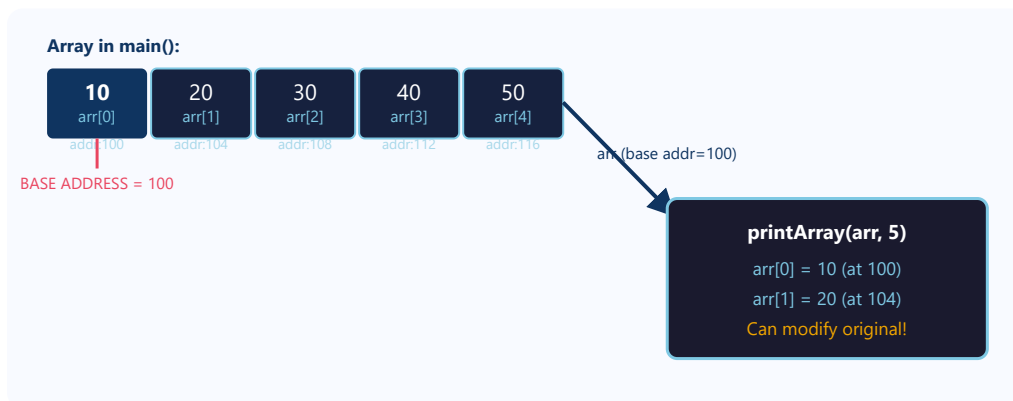
7.1 Syntax for Passing Arrays

FUNCTION PROTOTYPE WITH ARRAY

```
/* Method 1 – Array notation */
return_type function_name(int arr[], int size);

/* Method 2 – Pointer notation (equivalent) */
return_type function_name(int *arr, int size);
```

7.2 Array Passing Diagram

FIG 7.1 – ARRAY PASSED TO FUNCTION (BASE ADDRESS)

7.3 Programs Using Arrays as Parameters

C PROGRAM – PRINT ARRAY ELEMENTS USING FUNCTION

```
#include <stdio.h>

void printArray(int arr[], int n) {
    int i;
    printf("Array elements: ");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
```



```
int sumArray(int arr[], int n) {  
    int i, sum = 0;  
    for (i = 0; i < n; i++) sum += arr[i];  
    return sum;  
}  
  
int main() {  
    int a[] = {10, 20, 30, 40, 50};  
    printArray(a, 5);  
    printf("Sum = %d\n", sumArray(a, 5));  
    return 0;  
}
```

SECTION 08

Scope of Variables

DEFINITION

Scope refers to the region of the program where a variable is *visible* and can be accessed. Variables in C have different scopes based on where they are declared.

8.1 Local Variables

A **local variable** is declared inside a function or block { }. It is accessible only within that function/block and is destroyed when the function returns.

```
void func() {  
    int x = 10; /* Local to func() */  
    printf("%d\n", x);  
} /* x is destroyed here */  
int main() {  
    /* printf("%d\n", x); ERROR! x not visible here */  
    func(); return 0;  
}
```

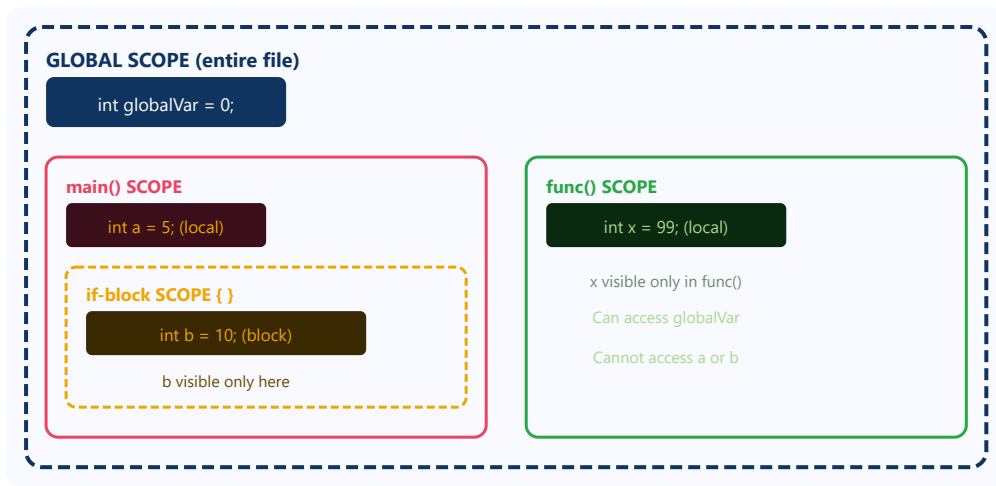
8.2 Global Variables

A **global variable** is declared outside all functions. It is accessible to all functions in the program and persists throughout the program's execution.

```
int count = 0; /* Global variable */  
void increment() { count++; }  
void display() { printf("Count = %d\n", count); }  
int main() {  
    increment(); increment(); display(); /* Count = 2 */  
    return 0;  
}
```

8.3 Scope Representation Diagram

FIG 8.1 – SCOPE OF VARIABLES IN C PROGRAM



SECTION 09

Lifetime of Variables

DEFINITION

Lifetime (also called *duration*) is the period during which a variable exists in memory — from its creation to its destruction.

Variable Type	Lifetime	Scope	Default Value
Local (auto)	During function execution only	Within function/block	Garbage
Global	Entire program execution	All functions in file	0
Static local	Entire program, but limited scope	Within function	0
Static global	Entire program execution	File scope only	0

Static Variable Example

STATIC VARIABLE – RETAINS VALUE ACROSS CALLS

```
void counter() {
    static int count = 0; /* Initialized only once */
    count++;
    printf("Call count: %d\n", count);
}

int main() {
    counter(); /* Output: Call count: 1 */
    counter(); /* Output: Call count: 2 */
    counter(); /* Output: Call count: 3 */
    return 0;
}
```

SECTION 10

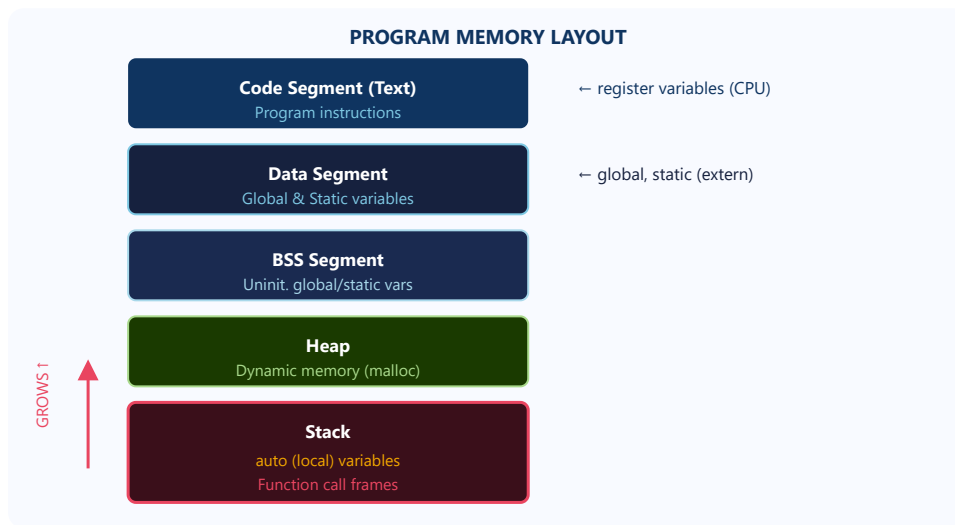
Storage Classes in C

DEFINITION

A **storage class** in C defines the *scope*, *visibility*, *lifetime*, and *default initial value* of a variable. C has four storage classes: **auto**, **register**, **static**, and **extern**.

10.1 Memory Allocation Diagram

FIG 10.1 – MEMORY SEGMENTS AND STORAGE CLASSES



10.2 The Four Storage Classes

1. auto (Default)

```
auto int x = 10; /* Same as: int x = 10; */  
/* Scope: Block, Lifetime: Until block ends, Default: Garbage */
```

2. register

```
register int i; /* Requests CPU register storage for fast access */  
for (i = 0; i < 1000; i++) { /* Faster loop counter */ }
```

3. static

```
static int count = 0; /* Initialized once, retains value */
```

4. extern

```
/* file1.c */  
int globalVar = 100;
```

```
/* file2.c */  
extern int globalVar; /* Declares variable defined in file1.c */
```

10.3 Storage Classes Comparison Table

Storage Class	Keyword	Scope	Lifetime	Default Value	Storage Location
Automatic	auto	Block/Function	Block duration	Garbage	Stack
Register	register	Block/Function	Block duration	Garbage	CPU Register
Static	static	Block (local) or File (global)	Program duration	0	Data Segment
External	extern	Entire program	Program duration	0	Data Segment

SECTION 11

Introduction to File Handling

DEFINITION

File handling in C refers to the ability of a program to create, open, read, write, and close files on secondary storage (disk). Unlike variables in memory (which are lost when the program ends), files *persist* data permanently.

11.1 Why File Handling?

- **Persistence:** Data remains after program terminates.
- **Large data:** Handle data too large to fit in memory.
- **Data sharing:** Multiple programs can access same file.
- **Batch processing:** Automate processing of large datasets.
- **Records:** Store and retrieve structured records (databases).

11.2 Types of Files

Type	Description	Access Method	Example
Text Files	Human-readable, newline-delimited	Sequential	.txt, .c, .csv
Binary Files	Machine format, compact	Random/Sequential	.exe, .dat, .jpg

11.3 The FILE Structure

DEFINITION

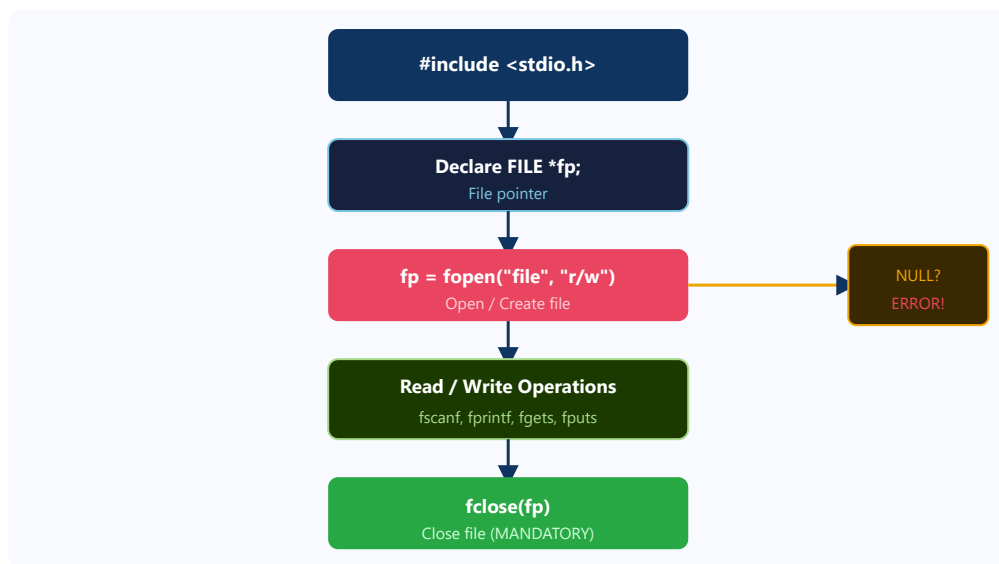
In C, a file is represented by a `FILE` pointer — a structure defined in `<stdio.h>` that stores information about the file (buffer, position, mode, error status). All file operations use a `FILE *` pointer.

FILE POINTER DECLARATION

```
FILE *filePointer;  
filePointer = fopen("filename.txt", "mode");
```

11.4 File Handling Process Flow – Diagram

FIG 11.1 – FILE HANDLING PROCESS FLOW



11.5 File Modes Table

Mode	Meaning	File Exists	File Not Found
"r"	Open for reading	Reads from beginning	Returns NULL
"w"	Open for writing	Truncates (erases) file	Creates new file
"a"	Open for appending	Writes to end	Creates new file
"r+"	Read and write	Reads/writes from start	Returns NULL
"w+"	Write and read	Truncates file	Creates new file
"a+"	Append and read	Reads any, writes to end	Creates new file
"rb"	Binary read	Opens binary for reading	Returns NULL
"wb"	Binary write	Truncates binary file	Creates new

SECTION 12

File Operations in C

12.1 Key File Functions

Function	Purpose	Syntax
fopen()	Open a file	FILE *fp = fopen("name","mode");
fclose()	Close a file	fclose(fp);
fprintf()	Write formatted text	fprintf(fp, "%d", x);
fscanf()	Read formatted text	fscanf(fp, "%d", &x);
fgets()	Read a string/line	fgets(str, n, fp);
fputs()	Write a string	fputs(str, fp);
fgetc()	Read a character	ch = fgetc(fp);
fputc()	Write a character	fputc(ch, fp);
feof()	Check end of file	while(!feof(fp))
rewind()	Go to beginning	rewind(fp);

12.2 Write to File – Program

```
C PROGRAM – WRITE DATA TO FILE

#include <stdio.h>

int main() {
    FILE *fp;
    int i, n = 5;
    int data[] = {10, 20, 30, 40, 50};

    fp = fopen("data.txt", "w"); /* Open for writing */
    if (fp == NULL) {
        printf("Error: Cannot open file!\n");
        return 1;
    }
    for (i = 0; i < n; i++) {
        fprintf(fp, "%d\n", data[i]); /* Write each integer */
    }
    fclose(fp); /* Always close the file */
    printf("Data written successfully!\n");
    return 0;
}
```


12.3 Read from File – Program

C PROGRAM – READ DATA FROM FILE

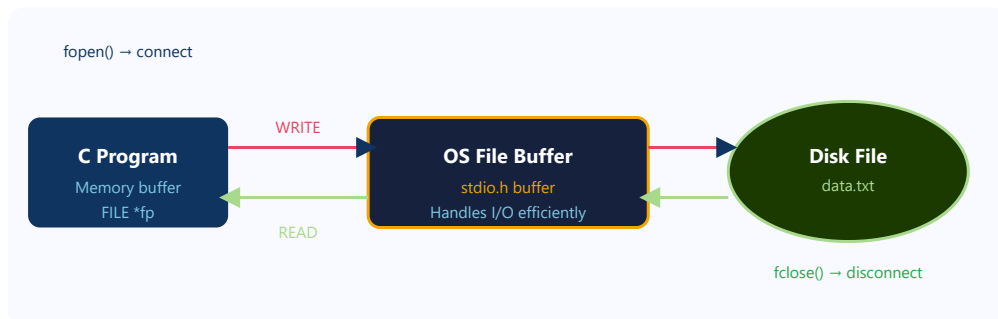
```
#include <stdio.h>

int main() {
    FILE *fp;
    int x;

    fp = fopen("data.txt", "r"); /* Open for reading */
    if (fp == NULL) {
        printf("File not found!\n");
        return 1;
    }
    printf("Contents of data.txt:\n");
    while (!feof(fp)) {
        fscanf(fp, "%d", &x);
        printf("%d\n", x);
    }
    fclose(fp);
    return 0;
}
```

12.4 File Read/Write Cycle – Diagram

FIG 12.1 – FILE READ/WRITE CYCLE



12.5 Copy File Program

C PROGRAM – COPY CONTENTS OF ONE FILE TO ANOTHER

```
#include <stdio.h>

int main() {
    FILE *src, *dest;
    char ch;

    src = fopen("source.txt", "r");
    dest = fopen("dest.txt", "w");
    while ((ch = fgetc(src)) != EOF)
        fputc(ch, dest);
    fclose(src); fclose(dest);
    printf("File copied successfully!\n");
    return 0;
}
```

SECTION 13

Questions & Answers

Part A – 2 Mark Questions

Q1 — 2 Marks

What is a function in C? Give its syntax.

Answer

A **function** in C is a self-contained block of statements that performs a specific task. It is identified by a unique name and can be called (invoked) from anywhere in the program. Functions are the building blocks of modular C programs.

Syntax:

```
return_type function_name(parameter_list) {  
    /* body of function */  
    return expression;  
}  
  
/* Example: */  
int add(int a, int b) { return a + b; }
```

Q2 — 2 Marks

What is the difference between function declaration and function definition?

Answer

A **function declaration** (prototype) tells the compiler the function's name, return type, and parameter types — it does not contain the body. It ends with a semicolon.

A **function definition** provides the actual implementation — it contains the complete code body enclosed in braces { }.

```
int add(int, int); /* Declaration - no body */  
int add(int a, int b) { return a+b; } /* Definition - has body */
```

Q3 — 2 Marks

What is call by value? How is it different from call by pointer?

Answer

In **call by value**, a copy of the actual argument is passed to the function. Modifications inside the function do NOT affect the original variable. In **call by pointer**, the memory address (pointer) of the variable is passed, so modifications inside the function DO affect the original variable.

```
/* Call by value: */ void f(int x) { x = 99; } /* original unchanged */  
/* Call by pointer: */ void f(int *x) { *x = 99; } /* original modified */
```

What is the scope of a variable?

Answer

Scope is the region of the program where a variable is visible and can be accessed. Variables declared inside a function are *local* (visible only within that function). Variables declared outside all functions are *global* (visible throughout the program). The scope of a variable is determined by where it is declared — inside a block { } means block scope.

```
int g = 10; /* Global scope - all functions can access */
void f() { int local = 5; } /* local only in f() */
```

What is the purpose of fopen() and fclose() in C?

Answer

fopen() opens a file and returns a FILE * pointer. It takes the filename and mode as arguments. If the file cannot be opened, it returns NULL. **fclose()** closes a previously opened file, flushes the buffer, and releases associated resources. Not closing a file can lead to data loss or file corruption.

```
FILE *fp = fopen("test.txt", "w"); /* open */
fprintf(fp, "Hello");
fclose(fp); /* close - mandatory */
```

What is a static variable? How is it different from a local variable?

Answer

A **static variable** declared inside a function retains its value between function calls. It is initialized only once (at program start) and exists throughout the program's lifetime, unlike a regular local (auto) variable which is created and destroyed each time the function is called.

```
void count() {
    static int n = 0; n++; printf("%d\n",n);
}/* 1, 2, 3 on successive calls */
```

Part B – 5 Mark Questions

Q7 — 5 Marks

Explain the four types of functions in C with examples and programs.

Answer

Functions in C are classified based on whether they receive arguments and whether they return values. There are four main types:

Type 1 – No Arguments, No Return Value: The function neither receives inputs from the caller nor sends back any output. It is suitable for printing menus or messages.

```
void displayMenu() {  
    printf("1. Add\n2. Subtract\n3. Exit\n");  
}  
  
int main() { displayMenu(); return 0; }
```

Type 2 – With Arguments, No Return Value: The function receives data from the caller but returns nothing. It processes data and may display results directly.

```
void printSquare(int n) {  
    printf("Square = %d\n", n * n);  
}  
  
int main() { printSquare(7); return 0; } /* Output: 49 */
```

Type 3 – No Arguments, With Return Value: The function takes no arguments but returns a computed value. Useful for reading user input.

```
int getInput() {  
    int n;  
    printf("Enter: "); scanf("%d", &n);  
    return n;  
}  
  
int main() { int x = getInput(); printf("%d\n",x); return 0; }
```

Type 4 – With Arguments and Return Value: The most powerful and commonly used type. The function receives arguments, processes them, and returns a result. Ideal for mathematical calculations.

```
int findMax(int a, int b) {  
    return (a > b) ? a : b;  
}  
  
int main() {  
    printf("Max = %d\n", findMax(15, 30)); /* Max = 30 */  
    return 0;  
}
```

Explain parameter passing using pointers in C with a program that swaps two numbers.**Answer**

In C, **call by pointer** (also called call by reference) passes the address of variables to a function. The function uses the pointer to directly access and modify the original variables in memory. This is essential when a function needs to modify more than one value of the caller, or when we want to avoid copying large data.

The pointer `*` operator is used to dereference (access the value at the pointed address), and `&` is used to get the address of a variable.

The following program swaps two numbers using pointers. When `swap(&x, &y)` is called, the addresses of `x` and `y` are passed. Inside `swap`, `*a` and `*b` refer to the original `x` and `y`, so changes persist after the function returns.

SWAP USING POINTERS – COMPLETE PROGRAM

```
#include <stdio.h>

void swap(int *a, int *b) {
    int temp;
    temp = *a; /* Save value at address a */
    *a = *b; /* Copy value at b to a */
    *b = temp; /* Copy saved value to b */
}

int main() {
    int x = 10, y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y); /* Pass addresses */
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}

/* Output:
Before swap: x = 10, y = 20
After swap: x = 20, y = 10 */
```

This works because `&x` gives the address of `x`. Inside `swap`, `*a` dereferences the pointer — accessing `x`'s memory location directly. The swap happens in the original memory, so after the function returns, `x` and `y` are actually swapped.

Q9 — 5 Marks

Write a C program using functions to find the factorial of a number and to check if it is prime.

Answer

The following program uses two separate functions: `factorial()` computes $n!$ recursively, and `isPrime()` checks primality by trial division. Both are called from `main()`.

COMPLETE PROGRAM – FACTORIAL AND PRIME CHECK

```
#include <stdio.h>

/* Function to compute factorial recursively */
long int factorial(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorial(n - 1);
}

/* Function to check prime: returns 1 if prime, 0 if not */
int isPrime(int n) {
    int i;
    if (n <= 1) return 0;
    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) return 0;
    }
    return 1;
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("Factorial of %d = %ld\n", num, factorial(num));
    if (isPrime(num))
        printf("%d is a Prime number.\n", num);
    else
        printf("%d is NOT a Prime number.\n", num);
    return 0;
}

/* Sample Output: (for input 5)
Factorial of 5 = 120
5 is a Prime number. */
```

Explain file handling in C with a program that writes student data to a file and reads it back.

Answer

File handling in C allows permanent storage of data. The standard library `<stdio.h>` provides functions like `fopen()`, `fprintf()`, `fscanf()`, and `fclose()` for file operations. A `FILE *` pointer acts as a handle to the file.

Steps: (1) Include `stdio.h`, (2) Declare `FILE` pointer, (3) Open file with `fopen()`, (4) Check for `NULL` (error), (5) Read/Write using `fprintf/fscanf`, (6) Close with `fclose()`.

STUDENT DATA FILE HANDLING PROGRAM

```
#include <stdio.h>
#include <string.h>

struct Student {
    int rollno;
    char name[30];
    float marks;
};

void writeStudents() {
    FILE *fp = fopen("students.txt", "w");
    if (!fp) { printf("Error!\n"); return; }
    fprintf(fp, "%d %s %.1f\n", 101, "Alice", 95.5);
    fprintf(fp, "%d %s %.1f\n", 102, "Bob", 88.0);
    fclose(fp);
    printf("Records written.\n");
}

void readStudents() {
    FILE *fp = fopen("students.txt", "r");
    int rno; char nm[30]; float m;
    printf("Roll Name Marks\n");
    while (fscanf(fp, "%d %s %f", &rno, nm, &m) == 3)
        printf("%-5d %-10s %.1f\n", rno, nm, m);
    fclose(fp);
}

int main() { writeStudents(); readStudents(); return 0; }
```

Explain storage classes in C with examples for each class.

Answer

Storage classes determine the scope, lifetime, visibility, and memory location of variables. C has four: auto, register, static, and extern.

auto is the default for local variables. **register** suggests CPU register storage for faster access. **static** preserves the value between function calls. **extern** makes a globally defined variable available across multiple files.

```
#include <stdio.h>

int extVar = 50; /* global / extern */

void demo() {
    auto int a = 1; /* auto: exists only in this call */
    register int r = 2; /* register: fast access */
    static int s = 0; /* static: retains value */

    s++;
    printf("a=%d r=%d s=%d ext=%d\n", a, r, s, extVar);
}

int main() {
    demo(); demo(); demo();
    return 0;
}

/* Output:
a=1 r=2 s=1 ext=50
a=1 r=2 s=2 ext=50 (s increments!)
a=1 r=2 s=3 ext=50 */
```


SECTION 14

Quick Revision Sheet – Unit V

✦ Function Syntax

```
return_type name(params) {
    /* body */
    return value;
}
```

- Prototype ends with ;
- Definition has a body { }
- void = no return value
- Parameters = formal args

✦ Parameter Passing Rules

- **Call by value:** copy passed → original safe
- **Call by pointer:** address passed → original modifiable
- Arrays always passed by base address
- Use & for address, * for dereferencing
- Pointers allow returning multiple values

✦ Scope Summary

- **Local:** inside function/block only
- **Global:** entire program
- **Static local:** function scope, survives calls
- Inner scope shadows outer scope
- Local variable default value = garbage
- Global variable default value = 0

✦ Storage Classes

Class	Default Val	Lifetime
auto	Garbage	Block
register	Garbage	Block
static	0	Program
extern	0	Program

✦ File Handling Steps

1. Include <stdio.h>
2. Declare FILE *fp;
3. Open: fp = fopen("name", "mode");
4. Check: if(fp == NULL)
5. Read/Write with fprintf/fscanf
6. Close: fclose(fp);

✦ File Modes

- "r" – read only
- "w" – write (truncate)
- "a" – append
- "r+" – read+write
- "w+" – write+read
- "rb"/"wb" – binary mode

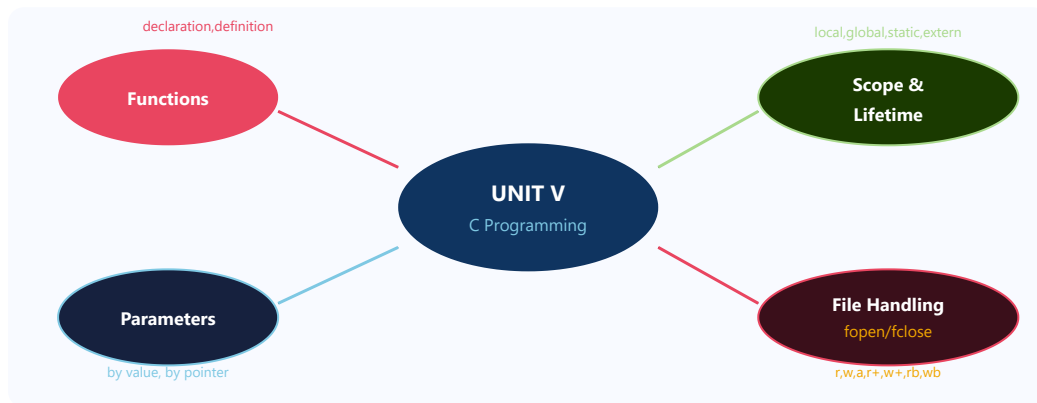
🚀 Key Functions Reference

- `fopen()` – open file
- `fclose()` – close file
- `fprintf()` – write formatted
- `fscanf()` – read formatted
- `fgetc/fputc` – char I/O
- `fgets/fputs` – string I/O
- `feof()` – end of file check
- `rewind()` – reset to start

🚀 Types of Functions

- Type 1: `void f()` – no arg, no ret
- Type 2: `void f(int)` – arg, no ret
- Type 3: `int f()` – no arg, ret
- Type 4: `int f(int, int)` – arg+ret
- Recursive: calls itself with base case

One-Page Concept Map



📖 Textbook References:

1. *The C Programming Language* – Brian W. Kernighan & Dennis M. Ritchie, Prentice Hall (1988). Chapters 4 (Functions), 8 (File I/O).
2. *Schaum's Outline of Programming with C* – Byron S. Gottfried, McGraw-Hill (1996). Chapters 7–9.
3. *Computing Fundamentals and C Programming* – E. Balagurusamy, McGraw-Hill (2008). Units on Functions and File Handling.
4. *Programming in C* – Rema Theraja, Oxford (2016). Chapter on Functions and I/O.
5. *C Programming: A Problem Solving Approach* – Forouzan, Gilberg, Prasad, Cengage (3rd Ed.). Chapters 9, 12.

Unit V – Functions & File Handling in C | B.Tech 1st Year Programming for Problem Solving

All the Best for Your Exams! 🎯